

Risk Mitigation and Analysis Document

Project: End-to-End Event Ticketing Platform

Course: Software Engineering II

Team Members: Behrad Hozouri
Amir Kalaki

40331411
40131038

1. Introduction

This document identifies potential architectural and operational failure points within the distributed event ticketing platform. Given the high-stakes nature of flash sales and concurrent user traffic, this analysis provides proactive engineering workarounds and architectural patterns to ensure high availability, data consistency, and a seamless user experience under extreme conditions.

2. Identified Architectural Risks and Mitigation Strategies

2.1. Flash Traffic Spikes & System Overload

Risk Description: During high-profile event drops, the platform will experience massive, sudden influxes of traffic. If the infrastructure cannot handle this concurrency, core services (especially the reservation engine) will crash, leading to HTTP 503 errors and complete system unavailability.

Impact Level	Probability	Engineering Workarounds & Mitigations
Critical	High	Virtual Waiting Room & Throttling: Implement a queueing system at the API Gateway layer to throttle incoming requests. Users are placed in a waiting room, allowing only a manageable volume of traffic to hit the

Impact Level	Probability	Engineering Workarounds & Mitigations
		reservation engine. • Horizontal Auto-Scaling: Utilize Kubernetes Horizontal Pod Autoscaler (HPA) to automatically spin up new Go-based container instances when CPU/Memory utilization exceeds predefined thresholds. • Read-Heavy Caching: Cache event catalogs and venue maps using Redis to drastically reduce database read queries during discovery phases.

2.2. Concurrency Control & Double-Booking (Race Conditions)

Risk Description: Multiple users may attempt to select and purchase the exact same seat simultaneously. Without strict synchronization, the database could process both transactions, resulting in a double-booked seat.

Impact Level	Probability	Engineering Workarounds & Mitigations
Critical	High	• Distributed Locks (Redis): When a user selects a seat, apply a distributed lock in Redis with a strict Time-To-Live (TTL). This ensures only one active session can

Impact Level	Probability	Engineering Workarounds & Mitigations
		hold the seat. • Optimistic/Pessimistic Database Locking: Use row-level locking (e.g., SELECT FOR UPDATE in PostgreSQL) or optimistic concurrency control (versioning) as a secondary validation layer before finalizing the database transaction. • Go Concurrency Features: Leverage Golang's channels and goroutines for safe, synchronous processing of reservation requests within memory boundaries before database writes.

2.3. External Payment Gateway Downtimes & Timeouts

Risk Description: The platform relies on third-party banking gateways for checkout. If the gateway times out, fails, or network communication drops, the user's payment state becomes unknown, and their selected seats remain indefinitely locked.

Impact Level	Probability	Engineering Workarounds & Mitigations
High	Medium	• Saga and Compensation Patterns: Implement distributed transaction

Impact Level	Probability	Engineering Workarounds & Mitigations
		management. If the payment domain confirms a failure or timeout, a compensation event is triggered to reverse the reservation and unlock the seat. • Automated TTL Expiration: Enforce a strict 10-minute TTL on temporary reservations. A background cron job or Redis keyspace notification will automatically reap expired locks and return seats to the available pool without manual intervention. • Idempotency Keys: Ensure all payment requests include idempotency keys so that retried requests (due to network drops) do not result in double charges.

2.4. Data Synchronization Bottlenecks (Inter-Service Network Partitioning)

Risk Description: In a microservices architecture, network partitions can occur. If the Billing domain successfully processes a payment but cannot communicate with the Notification or Ticket Issuance domain, the user will be charged without receiving their ticket.

Impact Level	Probability	Engineering Workarounds & Mitigations
High	Low-to-Medium	Asynchronous Messaging Brokers: Decouple domains using Kafka or RabbitMQ. The Billing domain writes an "Order Paid" event to a topic, and the Notification service consumes it asynchronously. This guarantees eventual delivery even if the Notification service is temporarily down. Dead Letter Queues (DLQ): If a message fails to process after multiple retries, it is routed to a DLQ for manual inspection and later replay, ensuring no ticket issuances are permanently lost. Transactional Outbox Pattern: Prevent dual-write failures by saving the payment state and the resulting event in the same local database transaction before publishing to the message broker.

3. Conclusion

By preemptively designing the system to handle failure as a natural state, the platform guarantees a high level of resilience. Combining distributed locks, asynchronous messaging, and container auto-scaling will ensure the ticketing platform meets its 99.99% uptime goal while protecting business integrity and user trust.